

Training Mechanics for Deep Learning

Computer Vision

Week 5 – Lecture 10

 Weight Initialization

 Learning Rate Scheduling

 Early Stopping

Recap & Motivation

Where we stand:

-  CNN architectures became deeper and highly capable.
-  Regularization (Dropout, Batch Norm) helped improve generalization.
-  But training a massive network still isn't guaranteed to succeed.

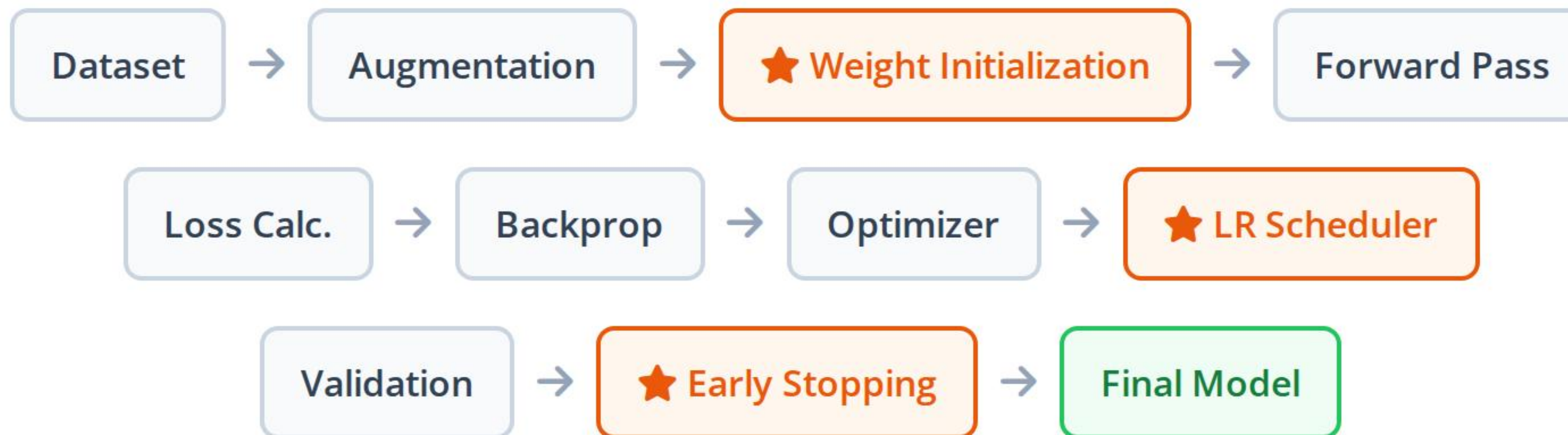
The Big Question:

"Can two identical neural networks produce completely different results on the exact same dataset?"

Yes. The difference lies entirely in the **training strategies**.

The Complete CNN Training Pipeline

Highlighting today's focus areas within the end-to-end workflow:



Why Weight Initialization Matters

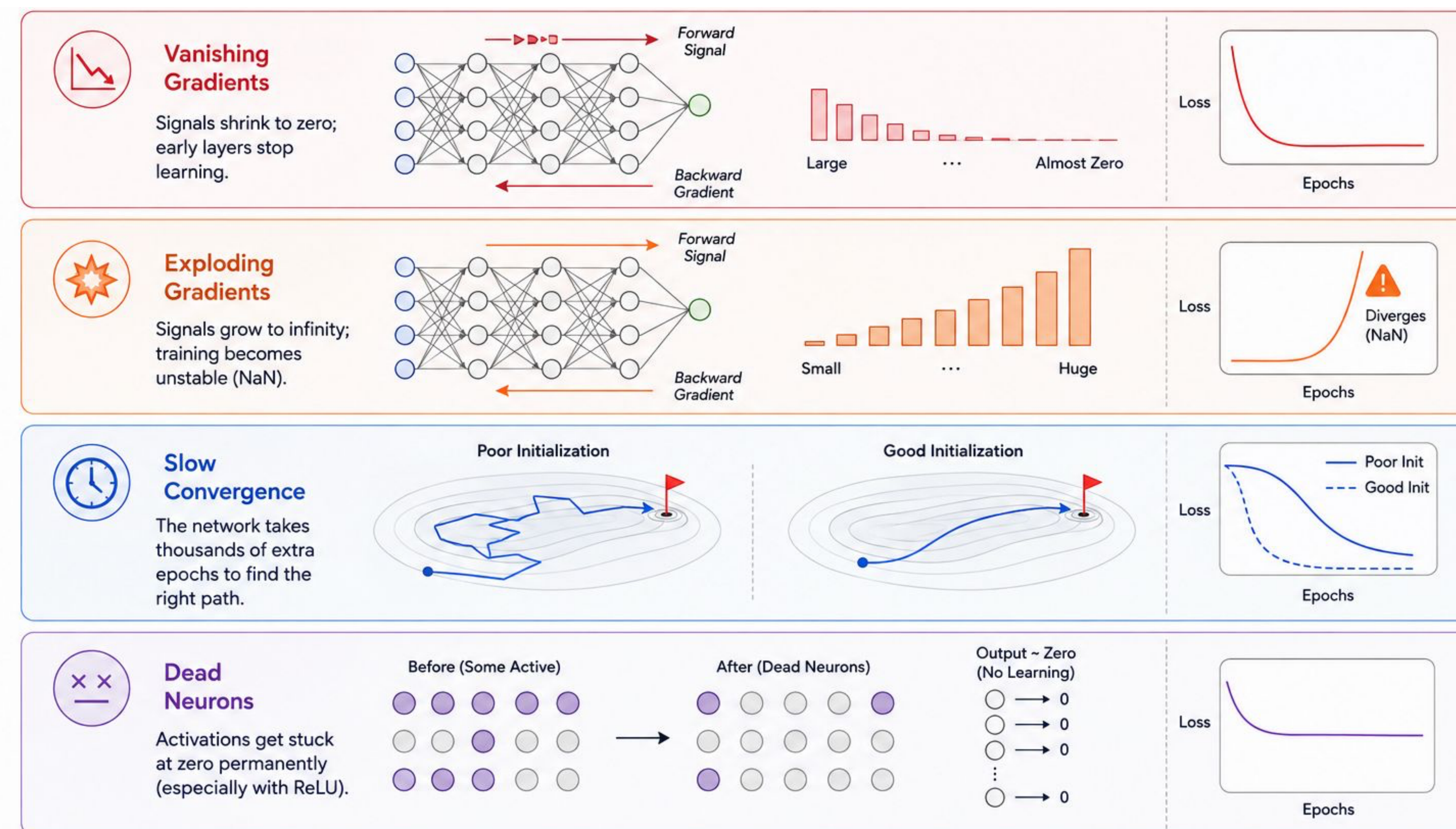
Neural networks begin with random weights. **Poor initialization can kill training before it begins:**

↓ **Vanishing Gradients:** Signals shrink to zero; early layers stop learning.

💧 **Exploding Gradients:** Signals grow to infinity; training becomes completely unstable (NaN).

Slow Convergence: The network takes thousands of extra epochs to find the right path.

💀 **Dead Neurons:** Activations get stuck at zero permanently (especially with ReLU).



Bad Initialization Examples



Very Small Weights

(e.g., all 0.0001)

Input $\rightarrow 0.1 \rightarrow 0.01 \rightarrow 0.001 \rightarrow 0$

Signals become weaker and vanish. No learning occurs.



Very Large Weights

(e.g., all 100)

Input $\rightarrow 100 \rightarrow 10,000 \rightarrow 1,000,000 \rightarrow$
NaN

Signals explode. The optimizer completely fails.



Balanced Initialization

(Statistically scaled)

Input $\rightarrow 1.2 \rightarrow 0.9 \rightarrow 1.1 \rightarrow 1.0$

Signals remain stable across all layers. Fast learning.

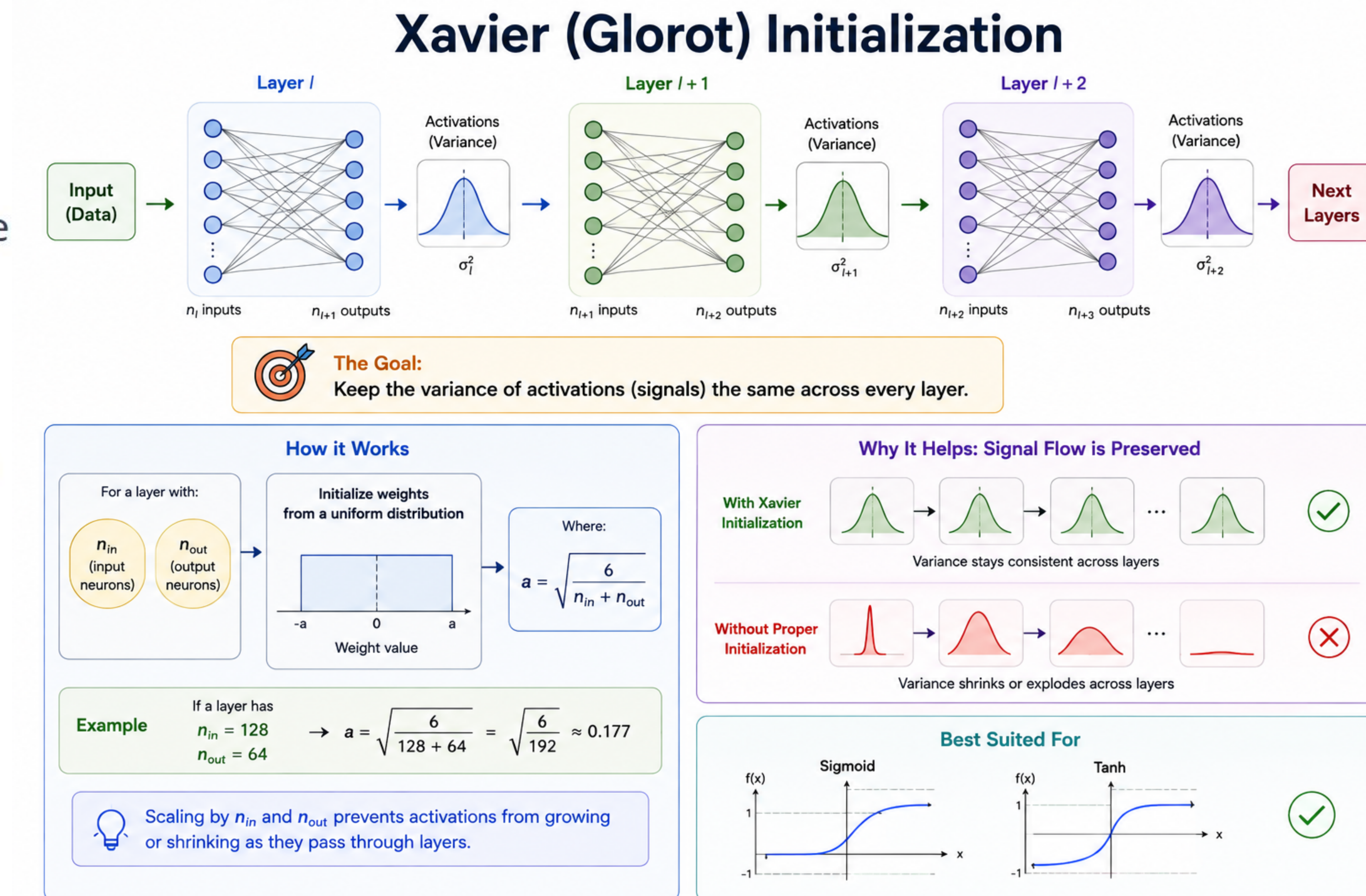
Xavier (Glorot) Initialization

Introduced by Xavier Glorot (2010).

 **The Goal:** Keep the variance of activations (signals) the same across every layer.

 **How it works:** It draws random weights from a distribution scaled by the number of input and output neurons for that specific layer.

✓ **Best suited for:** Networks using **Sigmoid** or **Tanh** activation functions.



He Initialization

He Initialization



Introduced by Kaiming He (2015).

⚠ **The Problem with Xavier:** Xavier breaks down when using ReLU, because ReLU zeroes out half of the data (all negative values), effectively halving the variance.

🔧 **The Fix:** He Initialization multiplies the Xavier variance by 2 to compensate for ReLU's destructive nature.

★ **The Standard:** This is the default initialization for **modern CNNs** using ReLU or Leaky ReLU.

Comparing Initialization Methods

Method	Target Activation	Core Mechanism	When to Use?
Standard Random	None	Picks from standard Gaussian ($N(0,1)$)	Rarely. Often leads to unstable training in deep nets.
Xavier (Glorot)	Sigmoid / Tanh	Scales variance by $1 / (\text{fan_in} + \text{fan_out})$	Shallow networks or RNNs using Tanh.
He (Kaiming)	ReLU / Leaky ReLU	Scales variance by $2 / \text{fan_in}$	Almost always for modern Deep CNNs (VGG, ResNet).

Takeaway: Match your initialization to your activation function!

Learning Rate: The Most Important Hyperparameter

The learning rate controls **how big of a step** the optimizer takes when updating weights to minimize loss.



Large Step (Too High)

The optimizer leaps over the lowest point of the valley. Loss bounces around erratically or explodes (diverges).



Small Step (Too Low)

The optimizer takes microscopic steps. Training takes weeks, or gets stuck in a shallow local minimum.



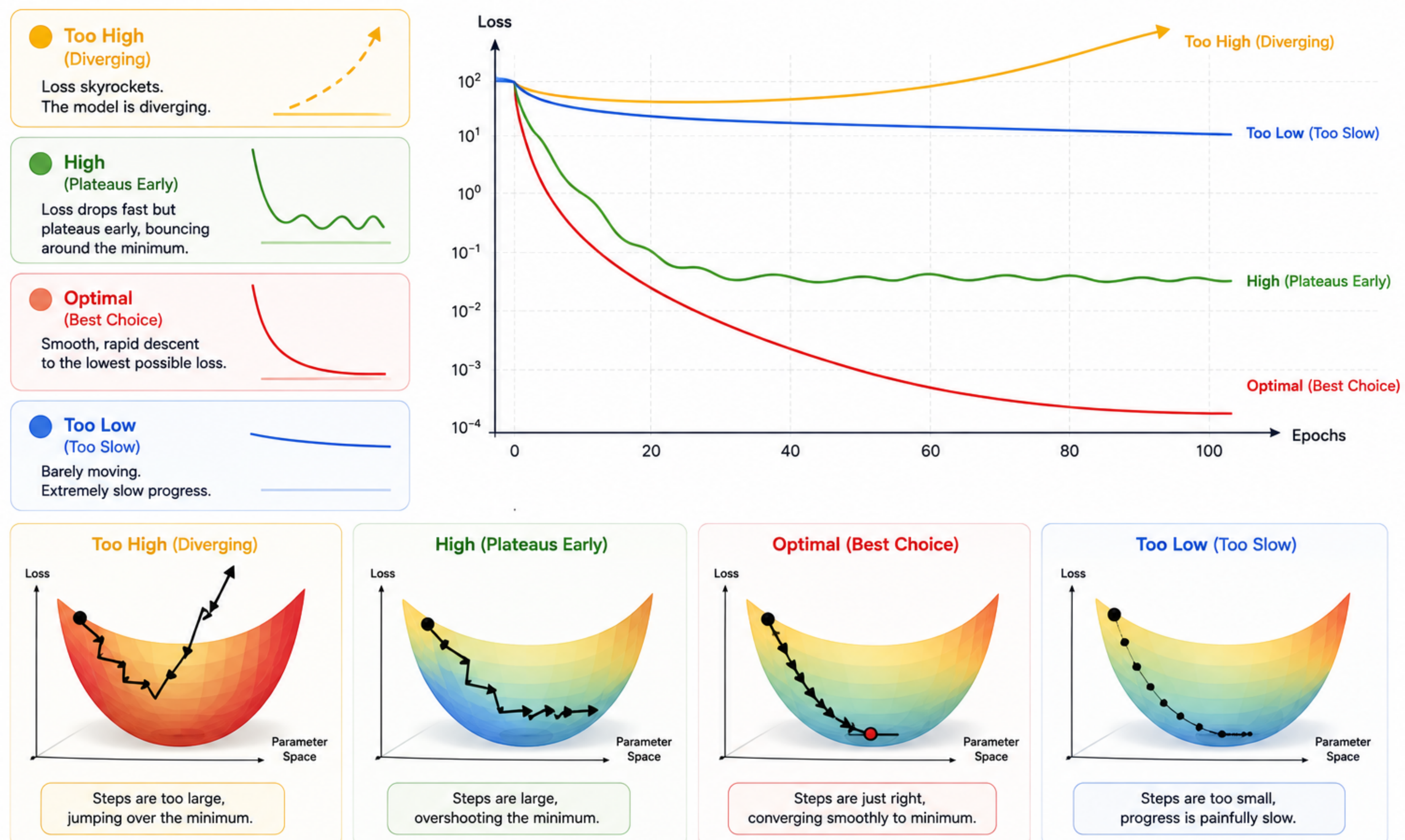
Appropriate Step

Smoothly and swiftly walks down into the deepest part of the valley. Fast and stable convergence.

Choosing the Right Learning Rate

-  **Yellow (Too High):** Loss skyrockets. The model is diverging.
-  **Green (High):** Loss drops fast but plateaus early, bouncing around the minimum.
-  **Red (Optimal):** Smooth, rapid descent to the lowest possible loss.
-  **Blue (Too Low):** Barely moving. Extremely slow progress.

Choosing the Right Learning Rate



Why Learning Rate Should Change

A static learning rate is rarely optimal for the entire training process.

🔍 Initially: Exploration

At the start of training, the model is far from the solution. A **larger learning rate** is needed to rapidly explore the landscape and escape local minimums.

🎯 Later: Fine-Tuning

As the model nears the optimal minimum, large steps will cause it to bounce out. A **smaller learning rate** allows it to carefully settle into the deepest part of the valley.

The Solution: Learning Rate Scheduling

Automatically decay (reduce) the learning rate as training progresses.

Common Learning Rate Schedulers

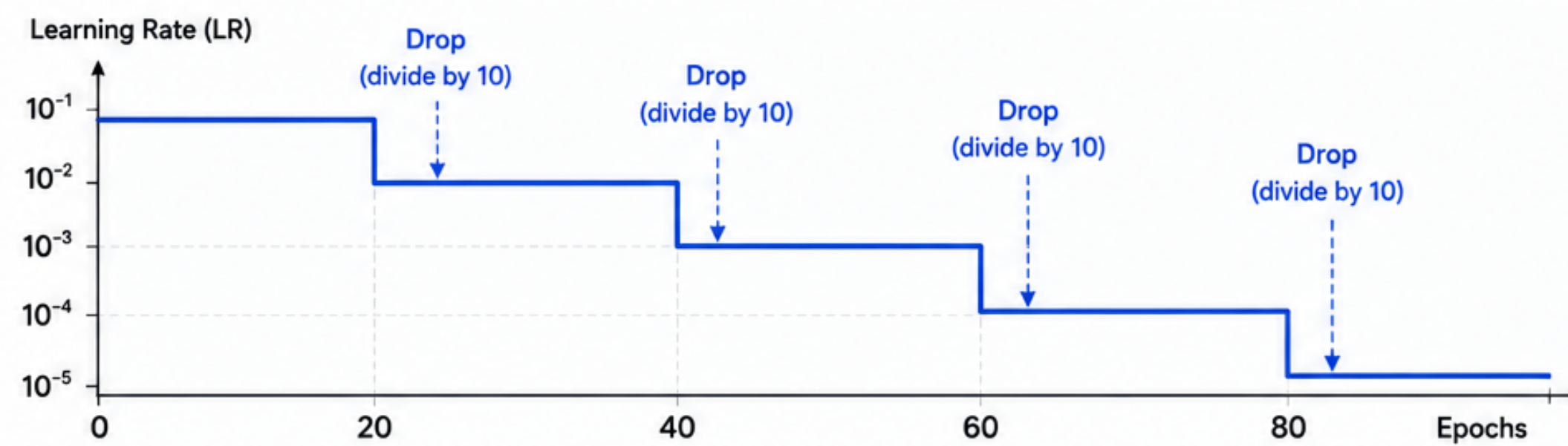
Common Learning Rate Schedulers



1. Step Decay

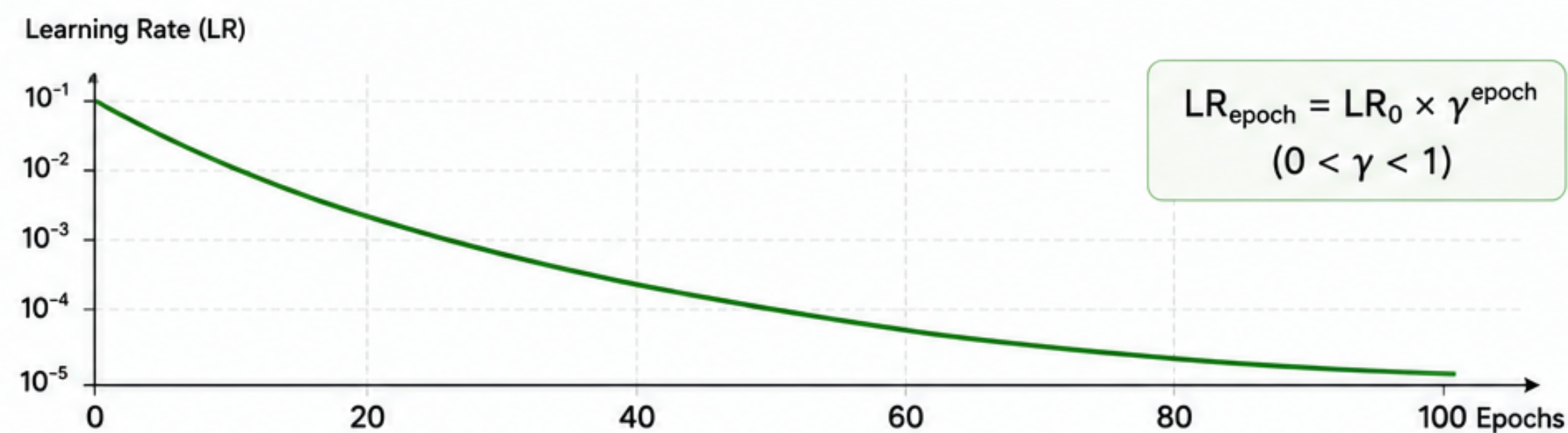
Drops the LR by a specific factor (e.g., divide by 10) every X epochs.

Very common and reliable.



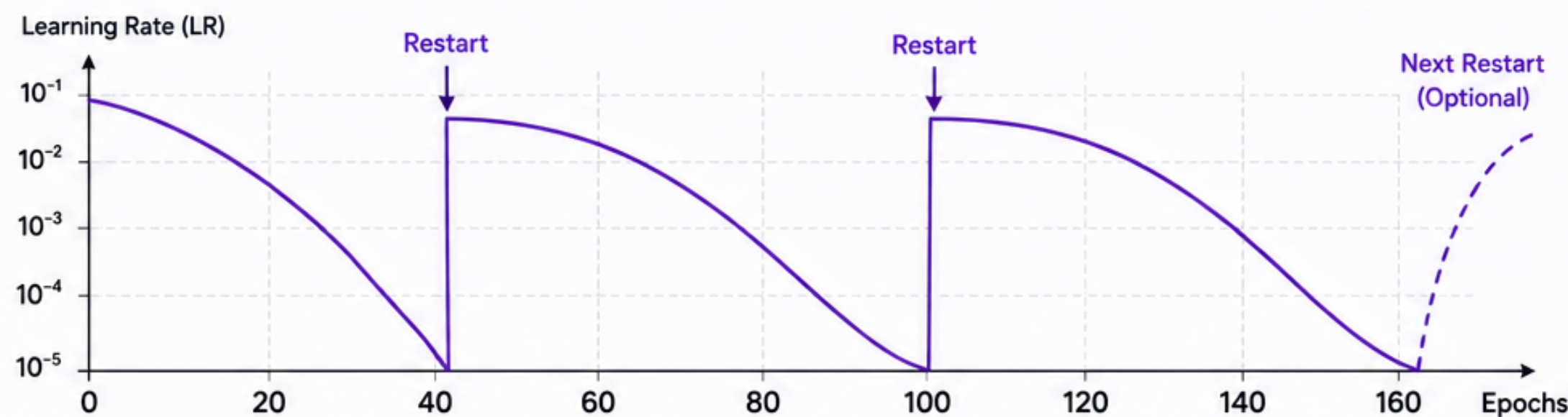
2. Exponential Decay

Continuously and smoothly decays the LR every single epoch.



3. Cosine Annealing

Smoothly drops the LR following a cosine curve, sometimes restarting to pop out of local minimums.



Step Decay: Drops the LR by a specific factor (e.g., divide by 10) every X epochs. Very common and reliable.

Exponential Decay: Continuously and smoothly decays the LR every single epoch.



Cosine Annealing: Smoothly drops the LR following a cosine curve, sometimes restarting to pop out of local minimums.

Learning Rate Scheduler Comparison

Scheduler	How it works	Advantage	Limitation
Step Decay	Drops LR by a factor at fixed epochs.	Simple, highly predictable.	Requires manual tuning of when to drop.
Cosine Annealing	Follows a cosine curve to zero.	Very smooth fine-tuning at the end.	Math can be rigid; doesn't adapt to loss.
Reduce-on-Plateau	Monitors Validation Loss. If loss stops improving, drops LR.	Highly adaptive. Only acts when the model actually needs help.	Can be triggered prematurely by noisy validation loss.

Reduce-on-Plateau is often the safest and most effective choice for general deep learning tasks.

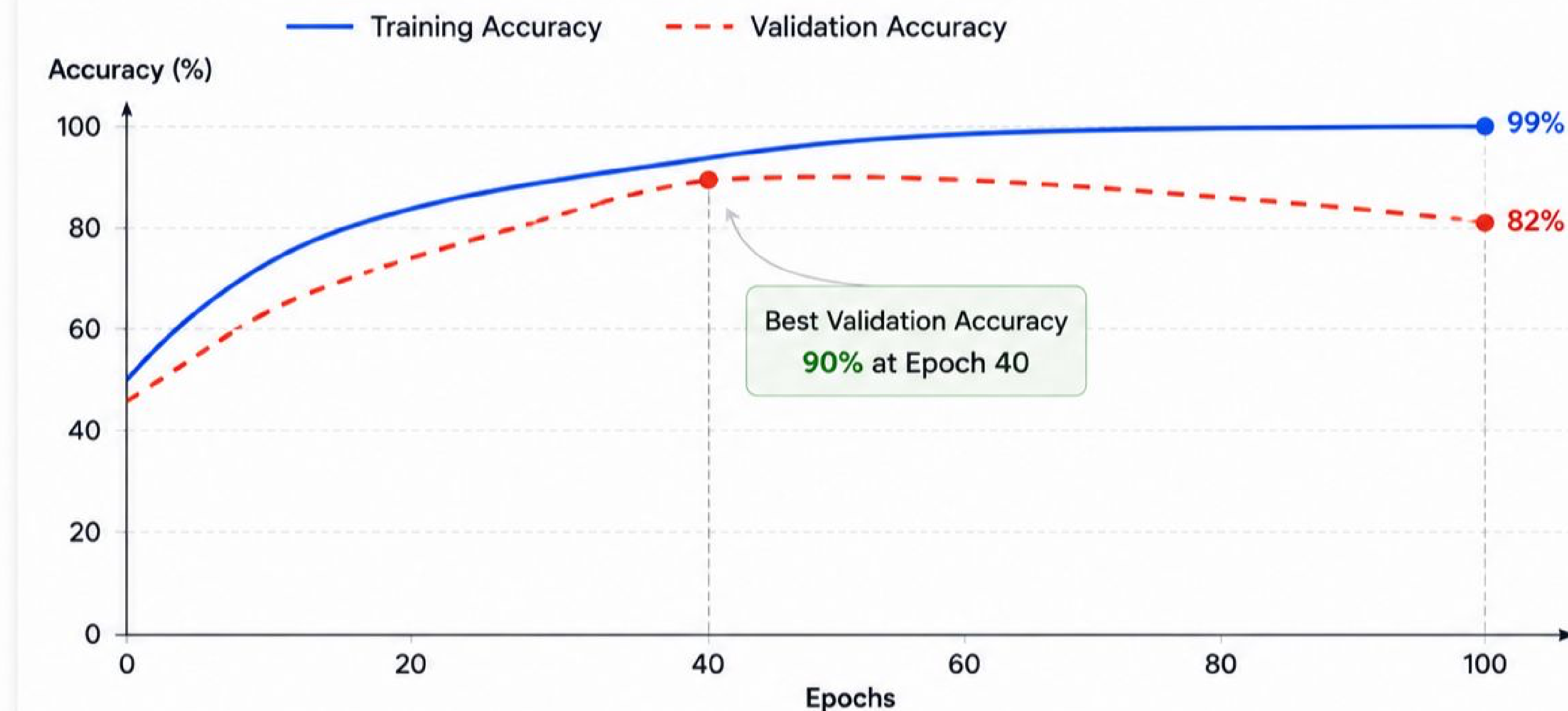
Early Stopping: Why Train Forever?

Consider this scenario:

- 🕒 You set your model to train for 100 epochs.
- 📈 At epoch 40, your validation accuracy hits 90%.
- ⚠️ By epoch 100, your training accuracy is 99%, but validation has dropped to 82% (Overfitting).

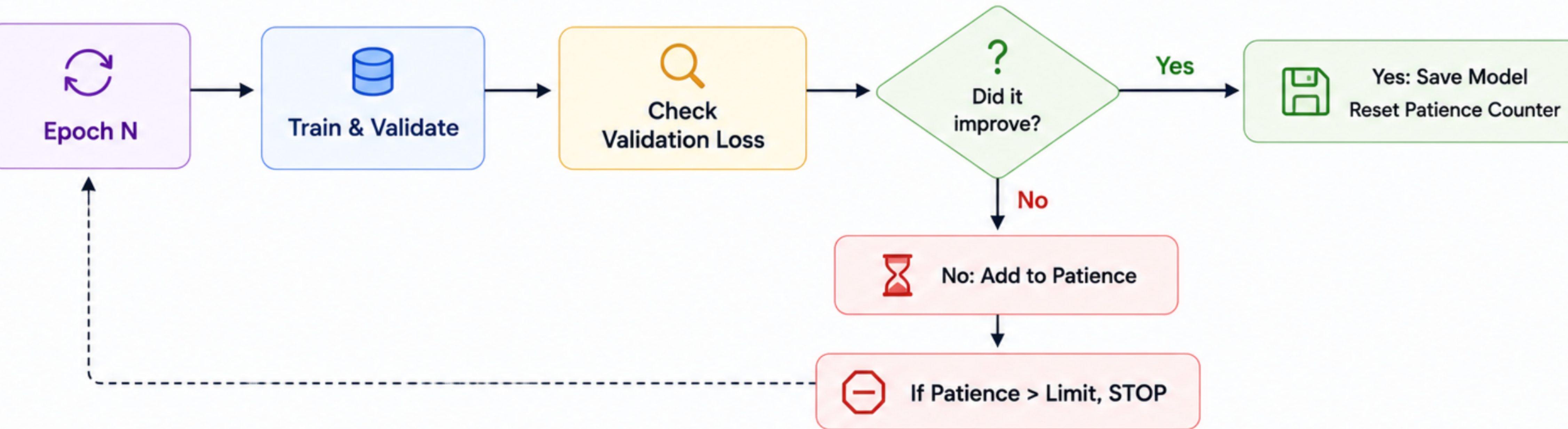
Question:

Why did we waste time calculating the last 60 epochs if the model was just getting worse?



Training longer increased training accuracy but hurt validation accuracy.
The model starts to overfit after epoch 40.

How Early Stopping Works



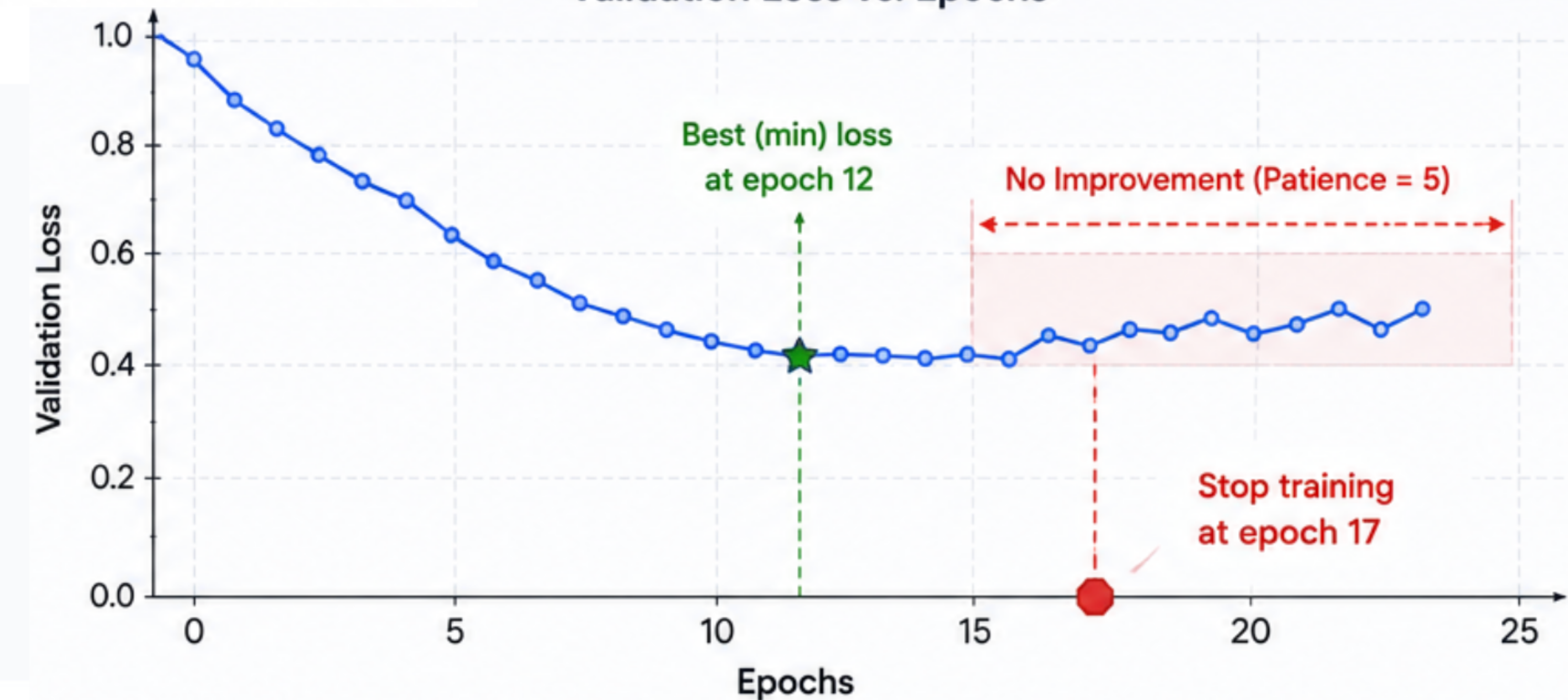
In this example:

- ★ Best validation loss at epoch 12
- After that, no improvement for 5 consecutive epochs
- Training stops at epoch 17
- 💾 Model from epoch 12 is kept

The "Patience" Parameter

We don't stop the moment loss goes up (it could just be noise). We wait for a set number of epochs (e.g., Patience = 5). If there is no improvement after 5 epochs, we kill the training process.

Validation Loss vs. Epochs



Benefits of Early Stopping



Prevents Overfitting

Ensures you never keep a model that has started memorizing the training data at the expense of validation data.



Saves Compute Time

If the model converges at epoch 30, it halts. You don't waste GPU hours running the remaining 70 epochs.

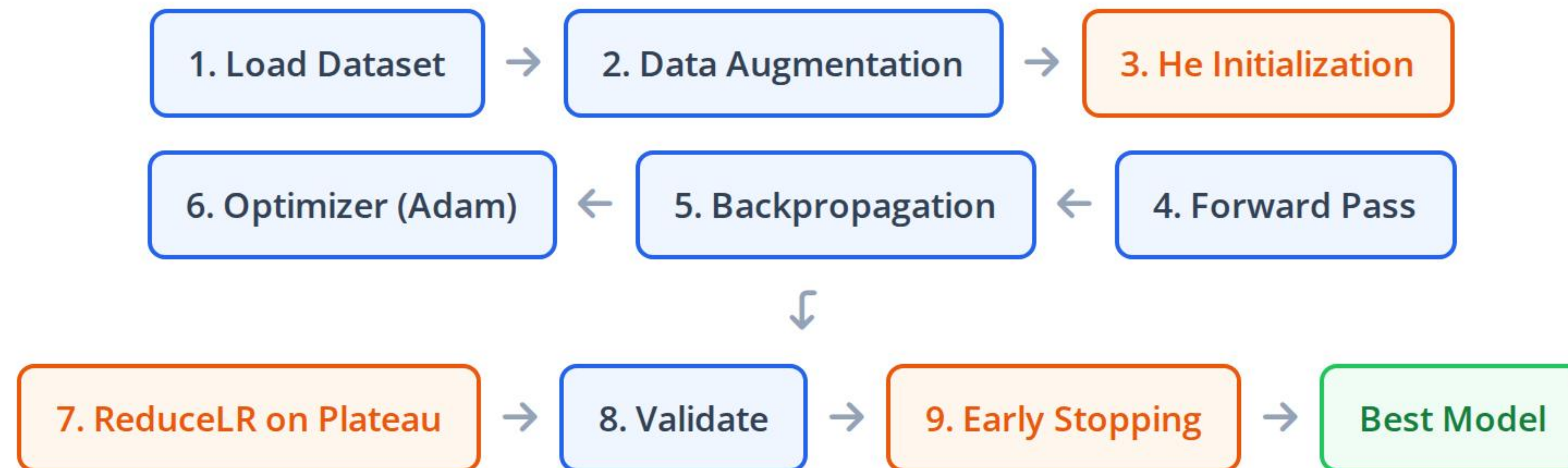


Automates Selection

You never have to manually guess how many epochs to train for. Set epochs to 1000, and let Early Stopping decide the perfect end point.

Putting Everything Together

The optimal, modern training workflow for a Convolutional Neural Network:



Best Practices for CNN Training

- ★ Always pair **ReLU** with **He Initialization**.
- ★ Use **Adam** or **AdamW** optimizer to start (usually defaults to a good LR like $1e-3$).
- ★ Use a **Reduce-on-Plateau** scheduler to automatically handle LR decay.
- ★ Set epochs to 500+ but use **Early Stopping** (patience ~10) to find the actual end point.

💡 Instructor Pro-Tip

"Don't try to manually guess hyperparameters. Build a robust pipeline that dynamically adjusts the learning rate and stops itself automatically based on validation metrics."

Common Training Mistakes

× Training Blind

Training only on the train set and not monitoring a validation set during the epoch loop. You have no idea if the model is overfitting.

× Static Learning Rates

Forcing a model to use $LR=0.01$ for 200 epochs. It will converge quickly but fail to fine-tune, bouncing around the local minimum permanently.

× Wrong Initialization

Using Xavier initialization with a deep ReLU network, leading to vanishing gradients and dead neurons in the early layers.

× Ignoring Saved Models

Running a script, watching validation accuracy peak at epoch 30, but the script finishes at epoch 100, leaving you with the overfitted epoch 100 weights.

Practical 4 Capstone Overview

In the upcoming lab, you will:

</> Build an end-to-end CNN image classifier in PyTorch.

🧪 Experiment with Random vs **He Initialization**.

Implement ReduceLROnPlateau to handle learning rate scheduling.

🕒 Code a custom **Early Stopping** class to save the best model weights.

➡ Expected Deliverables

- ✓ Plotted Training & Validation curves
- ✓ Graph showing LR decaying over time
- ✓ A best_model.pth weights file

Key Takeaways



Initialization

Always use He Initialization for networks using ReLU to ensure stable signal variance and prevent vanishing gradients.



Learning Rate

Use a high LR for initial exploration and a low LR for fine-tuning. Automate this using Schedulers like Reduce-on-Plateau.



Early Stopping

Never train blindly for fixed epochs. Monitor validation loss, save the best weights, and halt training when improvements stop.

Good architecture + Good regularization + **Smart Training Mechanics** = State of the Art Models.

Discussion Questions

1. Why can poor initialization entirely prevent a CNN from learning?
2. What happens to your loss curve if the learning rate is too large?
3. Why do we need to reduce the learning rate towards the end of training?
4. How does Early Stopping differ conceptually from Dropout?
5. Which scheduler monitors validation loss directly?

Next Week (Week 6 – Lecture 11)

Transfer Learning

Why not train from scratch?

Leveraging Pretrained Models

Fine-tuning vs Feature Extraction

(The most important technique in applied CV!)

Image Sources

600 × 400

<https://media.istockphoto.com/id/1387900612/vector/automation-data-analytic-with-robot-and-digital-visualization-for-business-strategy.jpg?s=612x612&w=0&k=20&c=L8Y457VbT9wXjE8R-Vj5aC8xG-U7B9h3P6k0v1R2Z0A=>

Source: www.istockphoto.com

600 × 400

https://media.springernature.com/lw685/springer-static/image/art%3A10.1007%2Fs11042-023-14532-7/MediaObjects/11042_2023_14532_Fig3_HTML.png

Source: link.springer.com

600 × 400

<https://prateekvjoshi.com/wp-content/uploads/2016/03/1-variance-formula.png>

Source: prateekvjoshi.com

600 × 400

<https://i.stack.imgur.com/g8O12.png>

Source: stats.stackexchange.com

600 × 400

<https://www.cs.umd.edu/class/spring2018/cmsc426/assets/cnn/lr.png>

Source: www.cs.umd.edu

600 × 400

https://d2l.ai/_images/output_lr-scheduler_774fb2_54_1.svg

Source: d2l.ai

Image Sources

600 × 400

https://upload.wikimedia.org/wikipedia/commons/thumb/1/1f/Early_stopping_machine_learning.svg/1200px-Early_stopping_machine_learning.svg.png

Source: en.wikipedia.org

600 × 400

[https://media.geeksforgeeks.org/wp-content/uploads/20231121113032/early-stopping-\(1\).png](https://media.geeksforgeeks.org/wp-content/uploads/20231121113032/early-stopping-(1).png)

Source: www.geeksforgeeks.org